Class - object => Variable types: ->

1. Object / instance -> object to invoke
2. Class variable (or) Static -> class will invoke

[Self.x -> public
Self._x -> protected
Self.__x -> private]

```
class Sample:
    def __init__(self):
        self.x = 10 # Public variable => Object Variable

    def modify(self):
        self.x += 1

s1 = Sample()
s2 = Sample()
print('x in s1 = ',s1.x)
print('x in s2 = ', s2.x)

s1.modify()
print('x in s1 = ',s1.x)
print('x in s2 = ', s2.x)
```

-> object variable

-> var is getting modified with object

-> object function (or) instance fun

s1 10 => 11

-> 11

-> 10

```
class Sample:
    x = 10 # Class variable

    @classmethod
    def modify(cls):
        cls.x += 1

s1 = Sample()
s2 = Sample()
print('x in s1 = ',s1.x)
print('x in s2 = ', s2.x)

s1.modify()
print('x in s1 = ',s1.x)
print('x in s2 = ', s2.x)
```

Class Variable

-> Class level

-> 11

NOT at Constructor

[def __init__(self)
    self.x = 10]

-> Object / instance

cls -> class

-> 10

-> 10

10 -> 11 ->

-> 11

-> 11 (10)

S2.modify()

Storage / memory of x

11

pri(s1.x) -> 12

prin(s2.x) -> 12

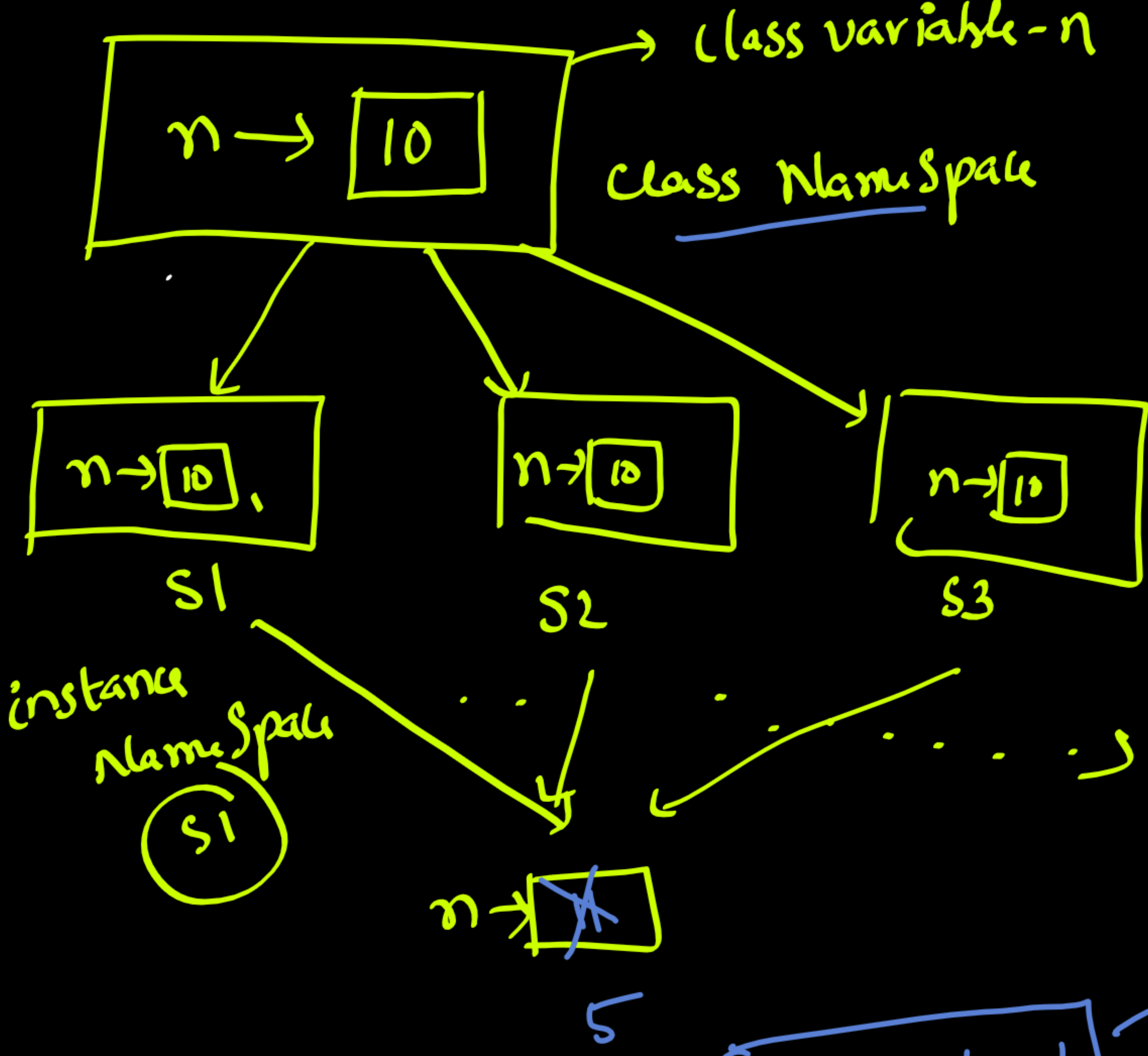
"Namespace"

x = 10 s1.modify

x -> [~~10~~ ~~11~~ 12] memory

Namespace

[Any object can modify cls variable impact it for all object.]



Class → will it create memory ?

NO

only when object is create
↳ memory.

→ Student

```
class Student:
    n = 10

print(Student.n)  # 10
Student.n += 1
print(Student.n)  # 11
```

10+1=11

S1 = Student
↳ S1.n = 5

self.x = 10

S1.x += 1 → 11

S2.x += 2 → 12

No impact on other object.

Types of methods

- ↳ instance methods
  - → Accessor [Getter methods]
  - → Mutator [Setter methods]
- ↳ Class methods
- ↳ Static methods

Bird: → Sel

Wing (2)

class animal:
    legs = 2

→ predefined

↳ class varies !

[Hor - 4
Hor - 2] ✗
↳ object

@Cla . .

class animal:
    def __init__ (sel, legs)
        self.leg =

Not comm